

Seminários — Técnicas de Inteligência Artificial para Jogos 2D

Os seminários têm como objetivo apresentar técnicas de Inteligência Artificial aplicadas ao desenvolvimento de jogos digitais 2D utilizando Python e a biblioteca Arcade. Cada grupo deverá desenvolver um exemplo prático funcional demonstrando a técnica escolhida, explicando seu funcionamento e apresentando visualmente o comportamento da IA durante a execução do programa.

Todos os exemplos desenvolvidos deverão obrigatoriamente seguir o modelo top-down de desenvolvimento de jogos 2D trabalhado em aula. Dessa forma, o projeto deve possuir uma organização adequada do código, utilizando classes, separação de responsabilidades, métodos específicos para atualização, desenho e controle do jogo, evitando códigos excessivamente lineares ou concentrados em um único bloco principal. O objetivo é que os trabalhos se aproximem da estrutura utilizada em projetos reais de desenvolvimento de jogos.

Os trabalhos deverão ser apresentados na data indicada para entrega no SIGAA. A próxima aula será destinada ao desenvolvimento dos exemplos, implementação do código e preparação do material necessário para a apresentação.

Cada seminário, com no **máximo 10 min.**, terá valor de 2,0 pontos. O foco principal da atividade será a correta implementação da técnica, a clareza da explicação durante a apresentação e a qualidade da demonstração prática desenvolvida pelo grupo.

ATENÇÃO: Somente quem participar da apresentação receberá nota.

Tema	Descrição	Desenvolvimento	Aluno(s)
1. Randomização Controlada	Técnica utilizada para criar comportamentos variáveis através de probabilidades e escolhas aleatórias controladas. Muito utilizada para evitar NPCs previsíveis.	EXCLUÍDA POR JÁ TER SIDO UTILIZADA DE FORMA GERAL NAS IMPLEMENTAÇÕES ANTERIORES.	
2. IA Baseada em Sensores	Técnica baseada na percepção do ambiente através de sensores simples, como distância, colisão ou áreas de detecção.	EXCLUÍDA POR JÁ TER SIDO UTILIZADA EM VÁRIAS IMPLEMENTAÇÕES ANTERIORES.	
3. Máquina de Estados (FSM)	Técnica que organiza o comportamento da IA em diferentes estados independentes. É uma das técnicas mais utilizadas em jogos comerciais.	EXCLUÍDA POR JÁ TER SIDO UTILIZADA DE FORMA GERAL NAS IMPLEMENTAÇÕES ANTERIORES.	
4. Campo de Visão (FOV)	Técnica utilizada para simular visão limitada através de um cone ou área de percepção visual. Muito utilizada em jogos stealth.	O exemplo deve implementar um cone de visão desenhado visualmente na tela, acompanhando a direção do NPC. O inimigo deve detectar o jogador apenas quando ele estiver dentro desse cone. O comportamento do NPC deve mudar ao detectar ou perder o jogador. Durante a apresentação deve ser explicado como funciona o algoritmo, as funções principais do código, como o cone é calculado, como ocorre a verificação de direção e por que o jogador só pode ser detectado dentro da área visível.	1. Mateus R. 2. Darley S.

5. Steering Behaviors	Técnica de movimentação inteligente baseada em vetores e forças de direção, produzindo movimentos mais naturais.	O trabalho deve implementar ao menos dois Steering Behaviors, como Seek, Flee, Wander ou Arrive. O movimento deve ser contínuo, suave e natural, sem movimentação em grid ou teletransporte. O comportamento atual deve aparecer na tela através de textos ou indicadores visuais. Durante a apresentação deve ser explicado como funciona o algoritmo, as funções principais do código, como os vetores são utilizados para calcular direção e velocidade, além de demonstrar a diferença visual entre os comportamentos implementados.	1. Julia 2. Milene
6. Raycasting / Linha de Visão	Técnica utilizada para verificar se existe visão direta entre dois pontos do cenário, considerando obstáculos.	O exemplo deve implementar paredes ou obstáculos reais no mapa e um sistema em que o NPC somente detecta o jogador se houver linha de visão livre. A linha de visão deve ser desenhada visualmente na tela. Quando o jogador estiver atrás de uma parede, o NPC deve perder a visão. Durante a apresentação deve ser explicado como funciona o algoritmo, as funções principais do código, como ocorre a verificação de obstáculos entre inimigo e jogador e como o raycasting foi implementado.	1. Júlio 2. Leonardo
7. A* (Pathfinding)	Algoritmo de busca de caminhos utilizado para navegação inteligente em mapas com obstáculos.	O trabalho deve implementar perseguição inteligente utilizando o algoritmo A*. O inimigo deve calcular automaticamente um caminho até o jogador contornando obstáculos. O caminho encontrado deve ser desenhado visualmente na tela utilizando linhas, blocos, pontos ou marcações. O jogador deve poder movimentar-se livremente enquanto o inimigo recalcula o caminho dinamicamente. Durante a apresentação deve ser explicado como funciona o algoritmo, as funções principais do código, o funcionamento geral do A*, o uso da heurística, a função das listas aberta e fechada e o que ocorre quando o objetivo muda de posição.	1. Diego B. 2. Cauã

8. Utility AI Simplificada	Técnica de tomada de decisão baseada em cálculo de utilidade para cada ação possível.	O exemplo deve implementar um NPC que escolha automaticamente entre diferentes ações, como atacar, fugir, patrulhar ou buscar vida, utilizando valores de utilidade calculados dinamicamente. Os valores das ações devem aparecer visualmente na tela durante a execução. O comportamento deve mudar automaticamente conforme a situação do jogo. Durante a apresentação deve ser explicado como funciona o algoritmo, as funções principais do código, como os valores são calculados e como a IA decide qual ação possui maior prioridade.	1. Mateus H 2. Marlon
9. Geração Procedural Simples	Técnica utilizada para gerar automaticamente conteúdo de jogos, como mapas, salas, obstáculos ou inimigos.	O trabalho deve implementar geração automática de algum elemento do jogo, como mapas, labirintos, posicionamento de inimigos, obstáculos ou itens. Cada execução deve gerar resultados diferentes. O cenário gerado deve permanecer jogável e visualmente compreensível. Durante a apresentação deve ser explicado como funciona o algoritmo, as funções principais do código, quais regras foram utilizadas na geração procedural e como o sistema evita gerar cenários inválidos ou impossíveis.	1. Rafael D. 2. Rafael c.
10. Flocking / Boids	Técnica utilizada para simular comportamento coletivo emergente em grupos de entidades. Muito utilizada para enxames, peixes e multidões.	O trabalho deve implementar as três regras clássicas de Boids: separação, alinhamento e coesão. As entidades devem mover-se coletivamente de forma natural, evitando colisões e mantendo o grupo organizado. O comportamento coletivo deve ser claramente visível na tela. Durante a apresentação deve ser explicado como funciona o algoritmo, as funções principais do código, explicar separadamente cada uma das três regras e demonstrar visualmente como cada comportamento influencia o movimento do grupo.	1. Robson 2. Ruvie

11. Follow the Leader	Técnica de movimentação em grupo onde múltiplos NPCs seguem um líder mantendo determinada formação ou distância. Muito utilizada em jogos com companheiros, tropas ou filas de inimigos.	O trabalho deve implementar um personagem líder controlado pelo jogador ou por movimento automático, enquanto outros NPCs seguem sua trajetória mantendo certa organização. O movimento deve ocorrer de forma contínua e visualmente compreensível. O exemplo deve mostrar claramente quem é o líder e como os seguidores acompanham sua movimentação. Durante a apresentação deve ser explicado como funciona o algoritmo, as funções principais do código, como os seguidores determinam a posição que devem ocupar e como o sistema evita sobreposição excessiva entre os personagens.	1. Talita 2. Geovane
12. Pursuit e Evasion	Técnica utilizada para prever o movimento futuro de um alvo, permitindo perseguições e fugas mais inteligentes. Diferencia-se da perseguição simples por considerar direção e velocidade do alvo.	O trabalho deve implementar um NPC capaz de prever para onde o jogador está se movendo antes de persegui-lo. Também pode ser implementado um comportamento de fuga inteligente. O exemplo deve mostrar visualmente a diferença entre perseguir a posição atual do jogador e prever sua posição futura. Devem ser desenhadas linhas indicando direção, previsão de movimento ou ponto futuro estimado. Durante a apresentação deve ser explicado como funciona o algoritmo, as funções principais do código e como velocidade e direção são utilizadas para prever a posição futura do alvo.	1. Daniel J. A 2. Camila 3. Fernando S.
13. Sistema de Cobertura e Fuga	Técnica utilizada para movimentação estratégica de NPCs buscando proteção contra o jogador ou perigos do cenário. Muito utilizada em jogos de tiro e sobrevivência.	O trabalho deve implementar um NPC capaz de procurar obstáculos ou áreas seguras ao detectar perigo. O personagem deve fugir do jogador e tentar posicionar-se atrás de paredes ou objetos do cenário. O exemplo deve apresentar visualmente quais posições estão sendo consideradas seguras, utilizando marcações, linhas ou áreas destacadas. Durante a apresentação deve ser explicado como funciona o algoritmo, as funções principais do código, como a IA identifica locais de cobertura e quais critérios são utilizados para decidir quando fugir ou buscar proteção.	1. João P. 2. Hewerton